

Employee Travel Desk (ETD)

A Microservices-Based Corporate Travel Management Platform

Technical Project Documentation

Cognizant FSE Business-Aligned Project

Java 21 | Spring Boot 3.5 | Spring Cloud | Angular 17 | JWT

5 Spring Boot microservices • 5 Angular micro-frontends

1. Project Overview

Employee Travel Desk (ETD) is a corporate travel-management platform that automates the complete business-trip lifecycle inside an organisation. It is built as a distributed system of five independent Spring Boot microservices on the backend and five matching Angular single-page micro-frontends on the front end, integrated through a shared, stateless JSON Web Token (JWT).

The platform serves three user roles, each with a distinct workflow:

Role	Responsibilities
HR	Manages employees and their grades, approves or rejects travel requests, and calculates the approved trip budget.
Employee	Raises travel requests, tracks their reservations, and submits post-trip reimbursement claims.
TravelDeskExe (Travel Desk Executive)	Uploads reservation bookings (flight / train / bus / cab / hotel) for approved trips and reviews / processes reimbursement claims.

End-to-end flow: Employee raises a request -- HR approves it and sets the budget -- TravelDeskExe books the reservations -- the Employee travels and submits invoices -- TravelDeskExe approves or rejects each reimbursement claim.

2. System Architecture

Each backend microservice is autonomous, owns its data, and exposes a REST API documented with OpenAPI / Swagger. Each Angular app maps one-to-one to a backend service and runs on its own port. The browser talks to every service directly (open CORS); there is no central API gateway in the current build.

Microservice	Port	Front-end app (port)	Responsibility
account-management	8081	account-management-app (4201)	Employees, grades, grade history; hosts the shared H2 TCP server (9092)
auth-service	8080	auth-app (4200)	Central login, token refresh, logout, JWT blacklist check
travel-planner	8082	travel-planner-app (4202)	Travel-request lifecycle and budget calculation
reservation-management	8083	reservation-management-app (4203)	Reservation booking and PDF storage / download
reimbursement-management	8084	reimbursement-management-app (4204)	Expense-claim submission and processing

Mandatory startup order

account-management (starts the H2 TCP server) -- auth-service (connects to it) -- travel-planner -- reservation-management -- reimbursement-management.

3. Technology Stack

Backend

Area	Technology / Version
Language and runtime	Java 21
Framework	Spring Boot 3.5.7, Spring Cloud 2025.0.0
Core modules	Spring Web (REST), Spring Security, Spring Data JPA
Inter-service calls	Spring Cloud OpenFeign (declarative HTTP clients)
Authentication	JWT 0.12.6 (HMAC-SHA256 signed JWT)
API documentation	springdoc-openapi (Swagger UI 2.7.0 / 2.2.0)
Boilerplate reduction	Lombok 1.18.40
Database	H2 (dev); MySQL connector wired but commented out
Build tool	Gradle (system Gradle; wrapper jar not committed)

Frontend

Area	Technology / Version
Framework	Angular 17.2 (NgModule-based)
Language	TypeScript 5.3
Reactive library	RxJS 7.8
UI / styling	Bootstrap 5 and Bootstrap Icons
HTTP / auth	HttpClient with auth and error interceptors; route + role guards
Tooling / test	Angular CLI 17, Karma + Jasmine

4. Backend Microservices in Detail

4.1 account-management (port 8081)

The master data store and security backbone. It owns the employee and grade tables and hosts the H2 database over a TCP server (port 9092) that auth-service also connects to.

- Entities: Employee, Grade (Grade-1/2/3), GradeHistory, RefreshToken, TokenBlacklist.
- Employee IDs use a sequence starting at 100000 (always 6 digits). Emails must end with @cognizant.com; passwords are BCrypt-12 hashed.
- Endpoints: GET/POST/PUT/DELETE /api/employees, GET /api/employees/me, GET /api/grades.
- Seeds three grades and three default users on first startup; cleans expired blacklist tokens.

4.2 auth-service (port 8080)

Centralised authentication for the whole system. It issues and refreshes tokens and is the single source of truth for the logout blacklist. It reads the shared H2 database but never owns the schema (ddl-auto = none).

- Issues a 1-hour access token and a 7-day refresh token on login; rotates the refresh token on use.
- Endpoints: POST /login, POST /auth/refresh, POST /auth/logout, GET /auth/blacklist/check.
- On logout, the refresh token is deleted and the access token is added to the blacklist.

4.3 travel-planner (port 8082)

Manages the travel-request lifecycle and budget calculation. Has its own H2 file database and fetches employee / grade data from account-management via Feign.

- Entities: TravelRequest, Location (8 seeded cities), TravelBudgetAllocation.
- Status flow: NEW -- APPROVED / REJECTED. Priority ONE/TWO/THREE caps trip duration (30 / 20 / 10 days).
- Budget = grade daily cap (Grade-1 Rs. 15,000, Grade-2 Rs. 12,500, Grade-3 Rs. 10,000) x number of days.
- Key endpoints: POST /new, GET /{hrId}/pending, PUT /{trId}/update, POST /calculatebudget, GET /my, GET /approved.

4.4 reservation-management (port 8083)

Lets a TravelDeskExe upload booking confirmations (PDF) for approved trips and lets employees track them. Enforces budget caps per reservation type.

- Reservation types: Flight, Train, Bus, Cab, Hotel. PDF max 1 MB; stored on disk, served on demand.
- Reservations budget = 70% of the approved budget; caps: transport 35%, cab 15%, hotel 50%.
- Date rules: train/bus exactly 1 day before the from-date; hotel on the from-date. One reservation per category.
- Endpoints: POST /add, GET /track/{travelRequestId}, GET /{id}, GET /{id}/download, GET /types.

4.5 reimbursement-management (port 8084)

Employees submit post-trip expense invoices with PDF proof; a TravelDeskExe approves or rejects each one. Enforces per-invoice ranges and daily cumulative limits.

- Types: Food, Water, Laundry, LocalTravel. PDF max 256 KB.
- Per-invoice: Food/Water Rs. 1,000-1,500; Laundry Rs. 250-500; LocalTravel up to Rs. 1,000.
- Daily caps: Food+Water combined 1,500/day; Laundry 500/day; LocalTravel 1,000/day.

- A reimbursement can only be raised once the travel request is APPROVED; only a TravelDeskExe can process it.
- Endpoints: POST /add, PUT /{id}/process, GET /{travelRequestId}/requests, GET /my, GET /types.

5. Front-end Applications

Five Angular 17 micro-frontends, one per backend, each with a consistent role-aware navigation bar. Tokens are passed between apps as URL parameters during cross-app navigation and seeded into each app's localStorage before route guards run.

App (port)	Primary role	Key pages
auth-app (4200)	All	Login
account-management-app (4201)	HR	Employee list, add employee, change grade
travel-planner-app (4202)	Employee / HR	Home dashboard, new request, my requests, pending requests, calculate budget
reservation-management-app (4203)	TravelDeskExe / Employee	Add reservation, view reservations, my reservations (track), details
reimbursement-management-app (4204)	Employee / TravelDeskExe	Submit claim, my claims, process claims (search)

6. Authentication and Authorization

- Stateless JWT signed with HMAC-SHA256 using a shared secret identical across all five services.
- Token claims: sub (email), role (HR / Employee / TravelDeskExe), iat, exp. Access token valid 1 hour; refresh token 7 days.
- Every protected service runs a JwtAuthFilter: it validates the signature and expiry locally and calls auth-service to check the blacklist on each request.
- Blacklist check is fail-open: if auth-service is unreachable, the check is skipped and only local validation applies.
- Spring Security maps the JWT role claim directly to an authority (no ROLE_ prefix); endpoints are guarded per role.

7. Database Design

All services use H2 in development (MySQL is wired but commented out). The data is split between one shared database and three independent ones.

Service	Database	Mode
account-management	account_management	H2 file + TCP server on 9092 (schema owner)
auth-service	account_management (shared)	H2 TCP client; ddl-auto = none
travel-planner	travel_planner	Own H2 file DB (AUTO_SERVER); ddl-auto = update
reservation-management	reservation_types	Own H2 file DB (AUTO_SERVER); ddl-auto = update
reimbursement-management	reimbursement_management	Own H2 file DB (AUTO_SERVER); ddl-auto = update

8. Inter-Service Communication (Feign)

Services call one another with Spring Cloud OpenFeign. A Feign request interceptor automatically forwards the caller's Authorization: Bearer token on every outgoing call, because the target service also validates the JWT.

Caller	Calls	Purpose
travel-planner	account-management, auth-service	Validate employee + grade; blacklist check
reservation-management	travel-planner, auth-service	Validate approved trip + budget; blacklist check
reimbursement-management	travel-planner, account-management, auth-service	Validate trip + dates; validate processor role; blacklist check

9. Key Business Rules

9.1 Grade changes (account-management)

- Grade can only be upgraded (a lower grade id = more senior), never downgraded.
- No grade change within 2 years of joining (earliest grade-history date).
- A grade can be changed at most once per year.
- A TravelDeskExe is always force-assigned to Grade-1 on creation.

9.2 Travel request and budget (travel-planner)

- Trip duration must not exceed the priority cap (ONE 30, TWO 20, THREE 10 days).
- Only the employee who is logged in can raise their own request; the approver must have the HR role.
- Budget is computed only after approval, from the grade daily cap times the number of days.
- Hotel rating by role: HR may book 5-STAR / 7-STAR; others 3-STAR / 5-STAR.

9.3 Reservation and reimbursement limits

- Reservation budget caps (of 70% of approved budget): transport 35%, cab 15%, hotel 50%.
- Reimbursement per-invoice and daily-cumulative limits enforced per expense type.
- Reservations and reimbursements are only allowed against an APPROVED travel request.

10. Default Seed Credentials (development)

Role	Email	Password	Employee ID
HR	admin.hr@cognizant.com	Admin@123	100000
TravelDeskExe	desk.exec@cognizant.com	Exec@123	100001
Employee	john.employee@cognizant.com	Employee@123	100002

11. Build, Run and Tooling

- Backend: gradle build / gradle bootRun per service (system Gradle; wrapper jar not committed).
- Frontend: npm start (ng serve) per app on ports 4200-4204.
- Swagger UI per service at /swagger-ui.html; H2 console at /h2-console (user sa, blank password).

12. Current Stage, Limitations and Future Enhancements

- Development stage: H2 file databases rather than MySQL; the JWT secret is hard-coded in each application.properties.
- No central API gateway: the browser calls each service directly over open CORS.
- Test coverage is limited to the default Spring context-load test in each service.
- Front-end shared code (navbar, config) is duplicated across the five apps (no shared library / Module Federation yet).
- Natural next steps: API gateway + service discovery, externalised secrets, MySQL, contract / integration tests, and a shared front-end library.

13. Resume Talking Points

Suggested project title

Employee Travel Desk (ETD) -- Microservices-Based Corporate Travel Management Platform

Tech line

Java 21, Spring Boot 3.5, Spring Cloud OpenFeign, Spring Security (JWT), Spring Data JPA, Angular 17, TypeScript, REST, OpenAPI / Swagger, H2 / MySQL, Gradle.

Achievement bullets (pick the 3-5 that fit your role)

- Designed and built a 5-service microservices backend (Spring Boot 3.5, Java 21) with autonomous databases and declarative Feign inter-service communication.
- Implemented centralised, stateless JWT authentication (HMAC-SHA256, 1-hour access + 7-day refresh tokens) with token rotation and a server-side logout blacklist checked on every request.
- Enforced role-based access control for three roles (HR, Employee, TravelDeskExe) using Spring Security authorities derived from JWT claims.
- Developed 5 Angular 17 micro-frontends with role-aware routing, HTTP interceptors for token injection, route / role guards, and secure cross-app token hand-off.
- Modelled complex domain logic: grade-change governance, priority-based trip-duration caps, grade-driven budget calculation, and per-category reservation / reimbursement budget limits.
- Documented all REST APIs with OpenAPI / Swagger and secured cross-service calls by forwarding the bearer token through a Feign request interceptor.

One-line version (for a skills-dense resume)

Built a full-stack microservices travel-management system (5 Spring Boot services + 5 Angular apps) with JWT-secured, role-based access and Feign-based inter-service communication.